

Topology Optimization

Lecture 7 – Gradients

Gradients

The adjoint method

pyadjoint

Course overview

No.	Topics
1	Introduction Function and vector spaces, Interpolation, L^2 projection
2	The Finite Element Method for PDEs 1: Poisson's Equation, The Finite Element Method, Properties of the Finite Element Method
3	The Finite Element Method for PDEs 2: Index notation, Linear Elasticity, Non-linear Poisson's equation, Non-linear Elasticity
4	Constrained Optimization 1: Optimality, KKT Theorem, Numerical Optimization Algorithms
5	Constrained Optimization 2: Method of Moving Asymptotes, MPI and PETSc
6	Introduction to Topology Optimization: The idea, Filtering and Projection, Python implementation
7	Gradients
8	Robust topology optimization, More on MPI
9	Wave equations
10	Acoustic waves and more on Robust Optimization
11	Penalty and Augmented Lagrangian Methods Stress-constraints in Topology Optimization 1
12	Stress-constraints in Topology Optimization 2
13	Light concentration using nanoparticle design
14	Heat conduction

Some mathematical prerequisites ✓ **DONE!**

Mathematical and numerical formulation of the Finite Element Method ✓ **DONE!**

Optimization Theory and numerical optimization methods ✓ **DONE!**

Basic Topology Optimization

Advanced Topology Optimization

Applications of Topology Optimization

Gradients

- The gradient of a function is, for example, used in Taylor expansion, here to 1st order

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0) \cdot (\mathbf{x} - \mathbf{x}_0)$$

- The change in function value is

$$\Delta f(\mathbf{x}) = f(\mathbf{x}) - f(\mathbf{x}_0) \approx \nabla f(\mathbf{x}_0) \cdot (\mathbf{x} - \mathbf{x}_0)$$

- The gradient of a function at a point $\mathbf{x}_0$ is thus directly related to how much the function change when going from $\mathbf{x}_0$ to a new (close by) point, $\mathbf{x}$.

- Example: Define $\mathbf{x} = (h, l)^T$ and let $f(\mathbf{x})$ measure the deflection of a beam as a function of the beam height h and length l . The gradient $\nabla f(\mathbf{x}_0)$ will tell us

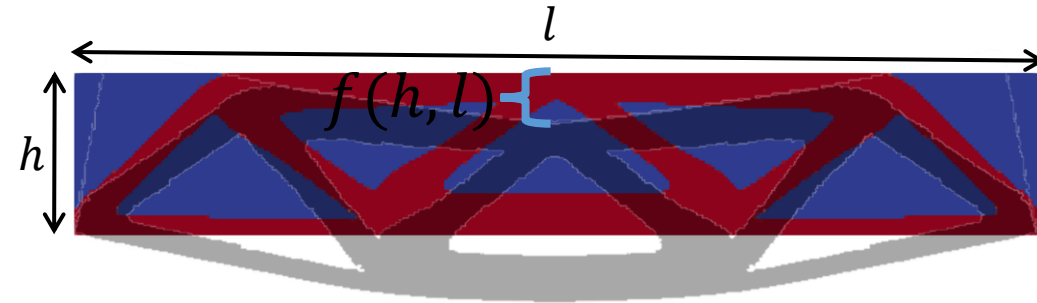
- $\nabla f(\mathbf{x}_0) = \begin{pmatrix} \text{"Relative change" in deflection when } h \text{ is changed from } h_0 \text{ to } h_0 + \Delta h \\ \text{"Relative change" in deflection when } l \text{ is changed from } l_0 \text{ to } l_0 + \Delta l \end{pmatrix}$ where Δh and Δl are small changes.

- Total change in f :

$$\Delta f(\mathbf{x}) = \nabla f(\mathbf{x}_0) \cdot (\mathbf{x} - \mathbf{x}_0) = \left. \frac{\partial f}{\partial h} \right|_{h_0, l_0} (h_0 + \Delta h - h_0) + \left. \frac{\partial f}{\partial l} \right|_{h_0, l_0} (l_0 + \Delta l - l_0) = \left. \frac{\partial f}{\partial h} \right|_{h_0, l_0} \Delta h + \left. \frac{\partial f}{\partial l} \right|_{h_0, l_0} \Delta l$$

= "Change in deflection when h is changed from h_0 to $h_0 + \Delta h$ " + "Change in deflection when l is changed from l_0 to $l_0 + \Delta l$ "

- Calculating gradients is also known as sensitivity analysis, as the gradient-components measures how "sensitive" a function is to a change in the corresponding variable.



Gradients

- To use gradient-based optimization, such as MMA, we need to calculate the gradients to all functions in the problem.

- In the topology optimization from last week, we had

Used directly in MMA

$$\min_{\rho(\mathbf{r})} F(\rho(\mathbf{r})) = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} d\Omega$$

$$\text{such that } \sigma_{ij,j} = f_i \text{ in } \Omega, \quad \sigma_{ij} n_j = T_i \text{ on } \partial\Omega^N, \quad u_i = u_i^D \text{ on } \partial\Omega^D$$

$$\epsilon_{ij} = \frac{1}{2} (u_{i,j} + u_{j,i})$$

$$\sigma_{ij} = \lambda(E) \epsilon_{kk} \delta_{ij} + 2\mu(E) \epsilon_{ij}$$

$$-r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} = \rho, \quad \frac{\partial \tilde{\rho}}{\partial \mathbf{n}} = 0$$

$$\bar{\rho} = H(\beta, \eta, \tilde{\rho}), \quad H(\beta, \eta, \tilde{\rho}) = \frac{\tanh(\beta\eta) + \tanh(\beta(\tilde{\rho} - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))}$$

$$E(\mathbf{r}) = E_{\min} + (E_{\text{material}} - E_{\min}) \bar{\rho}^p(\mathbf{r})$$

$$\int_{\Omega} \bar{\rho}(\mathbf{r}) d\Omega \leq V_{\max}$$

$$0 \leq \rho \leq 1$$

- To use MMA, we need to be able to differentiate the cost function F and the volume constraint function with respect to the DOFs of the design field, ρ .

Volume Constraint Gradient

- Let's first imagine we do not do filtering and projection and consider the simpler volume constraint $\int_{\Omega} \rho d\Omega \leq V_{max}$.

Side length 10
 $V_f = 0.25$

- We scaled by $1/V$ and write

$$g(\rho) = \frac{1}{V} \left(\int_{\Omega} \rho d\Omega - V_{max} \right) = \frac{1}{V} \int_{\Omega} (\rho - V_f) d\Omega \leq 0 ,$$

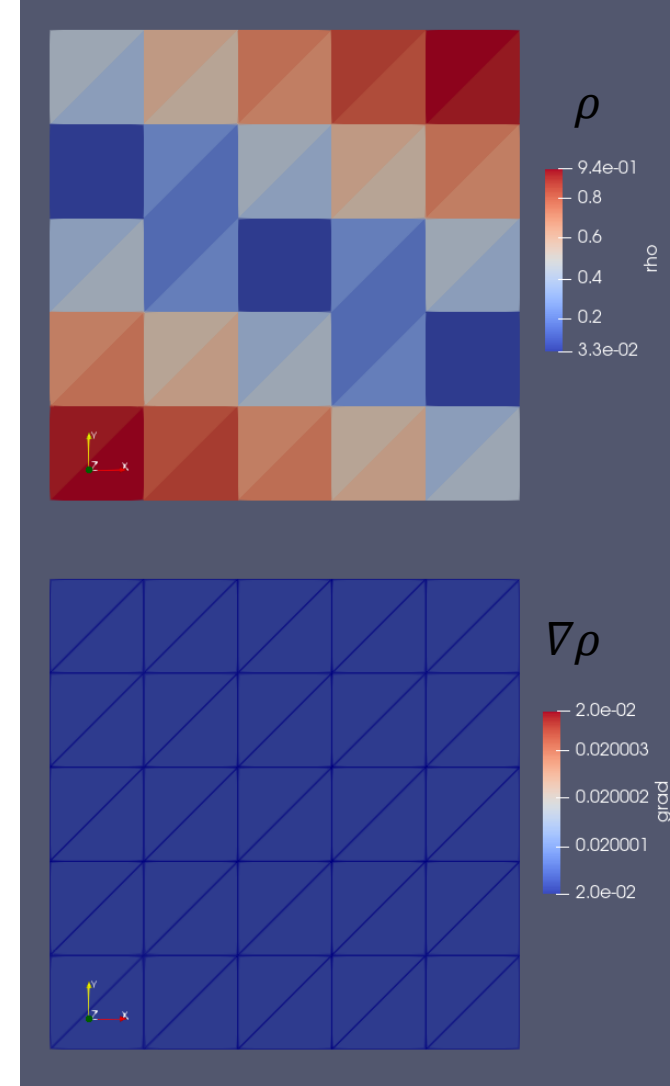
$$V_f = \frac{V_{max}}{V} , \quad V = \int_{\Omega} 1 d\Omega , \quad V_{max} = \int_{\Omega} V_f d\Omega$$

- We must calculate $\frac{dg}{d\rho_i}$, where ρ_i are the Degrees of Freedom associated with the function space where ρ is defined.
- We have used the space of piecewise constant functions for ρ .
- The gradient components are

$$(\nabla g(\rho))_i = \frac{dg}{d\rho_i} = \frac{\partial}{\partial \rho_i} \left(\frac{1}{V} \int_{\Omega} (\rho - V_f) d\Omega \right) = \frac{1}{V} \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} d\Omega$$

- Since ρ is piecewise constant and the constants are ρ_i , $\frac{\partial \rho}{\partial \rho_i} = 1$ inside mesh element i and $\frac{\partial \rho}{\partial \rho_i} = 0$ outside mesh element i . The integral thus becomes

$$(\nabla g(\rho))_i = \frac{1}{V} \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} d\Omega = \frac{1}{V} \int_{\Omega_i} \frac{\partial \rho}{\partial \rho_i} d\Omega = \frac{1}{V} \int_{\Omega_i} 1 d\Omega = \frac{\text{Area}(\Omega_i)}{V}$$



In the example there are 50 triangles of equal area, so $V = 50 \text{Area}(\Omega_i)$ and $(\nabla g(\rho))_i = \frac{1}{50} = 0.02$

Volume Constraint Gradient

- Let's first imagine we do not do filtering and projection and consider the simpler volume constraint $\int_{\Omega} \rho d\Omega \leq V_{max}$.

Side length 10
 $V_f = 0.25$

- We scaled by $1/V$ and write

$$g(\rho) = \frac{1}{V} \left(\int_{\Omega} \rho d\Omega - V_{max} \right) = \frac{1}{V} \int_{\Omega} (\rho - V_f) d\Omega \leq 0 ,$$

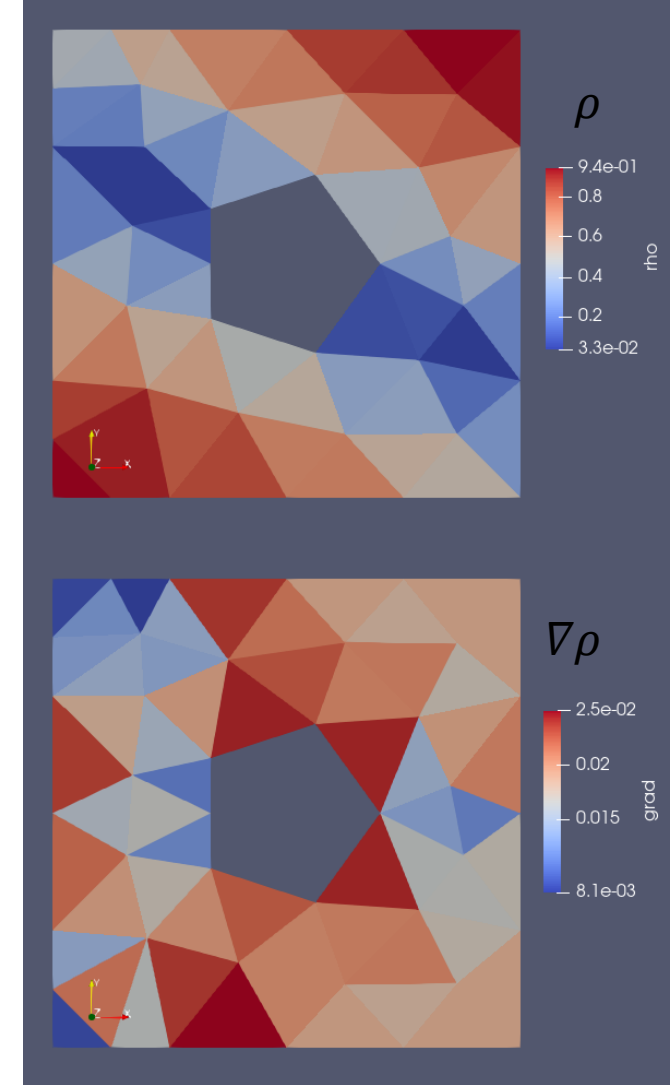
$$V_f = \frac{V_{max}}{V} , \quad V = \int_{\Omega} 1 d\Omega , \quad V_{max} = \int_{\Omega} V_f d\Omega$$

- We must calculate $\frac{dg}{d\rho_i}$, where ρ_i are the Degrees of Freedom associated with the function space where ρ is defined.
- We have used the space of piecewise constant functions for ρ .
- The gradient components are

$$(\nabla g(\rho))_i = \frac{dg}{d\rho_i} = \frac{\partial}{\partial \rho_i} \left(\frac{1}{V} \int_{\Omega} (\rho - V_f) d\Omega \right) = \frac{1}{V} \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} d\Omega$$

- Since ρ is piecewise constant and the constants are ρ_i , $\frac{\partial \rho}{\partial \rho_i} = 1$ inside mesh element i and $\frac{\partial \rho}{\partial \rho_i} = 0$ outside mesh element i . The integral thus becomes

$$(\nabla g(\rho))_i = \frac{1}{V} \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} d\Omega = \frac{1}{V} \int_{\Omega_i} \frac{\partial \rho}{\partial \rho_i} d\Omega = \frac{1}{V} \int_{\Omega_i} 1 d\Omega = \frac{\text{Area}(\Omega_i)}{V}$$



As the mesh element areas vary, so does $(\nabla g(\rho))_i$.

Volume Constraint Gradient

- The gradient of $g(\rho) = \frac{1}{V} \int_{\Omega} (\rho - V_f) d\Omega$ is

$$(\nabla g(\rho))_i = \frac{dg}{d\rho_i} = \frac{\partial}{\partial \rho_i} \left(\frac{1}{V} \int_{\Omega} (\rho - V_f) d\Omega \right) = \frac{1}{V} \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} d\Omega = \frac{1}{V} \int_{\Omega_i} 1 d\Omega = \frac{\text{Area}(\Omega_i)}{V}$$

- The gradient vector is a vector where each element is the corresponding mesh element area divided by V .
- This can be calculated in FEniCS using

```
Vd=FunctionSpace(mesh,"DG",0) # Define DG0 function space.
rho=interpolate(Expression('abs(sin(0.1*x[0]+0.2*x[1]))', degree=0) , Vd) # To get some numbers in the function rho.
rho.rename('rho','rho') # Rename to get a nice name in paraview.
drho=TestFunction(Vd) # Create a test function in Vd.
Vf=0.25; V=assemble(Constant(1)*dx(domain=mesh)) # Set volume fraction and calculate total volume (area in 2D).
grad_g=derivative( (rho-Vf)/V , rho , drho ) # UFL expression for the derivative.
grad_g_vector=assemble( grad_g*dx ) # Calculates the actual values and store them in a FEniCS vector.
```

- In the example of the square, `print( grad_g_vector.get_local() )` will show an array of elements all being 0.02, while in the example with the hole, the components have different values.

- The UFL expression ‘`derivative( f , u , du)`’ calculates the (Gateaux) derivative of ‘f’ with respect to the coefficients (DOFs) in ‘u’ in the direction ‘du’:

$$\text{derivative}(f , u , du) = \lim_{\tau \rightarrow 0} \frac{d}{d\tau} f(u + \tau du)$$

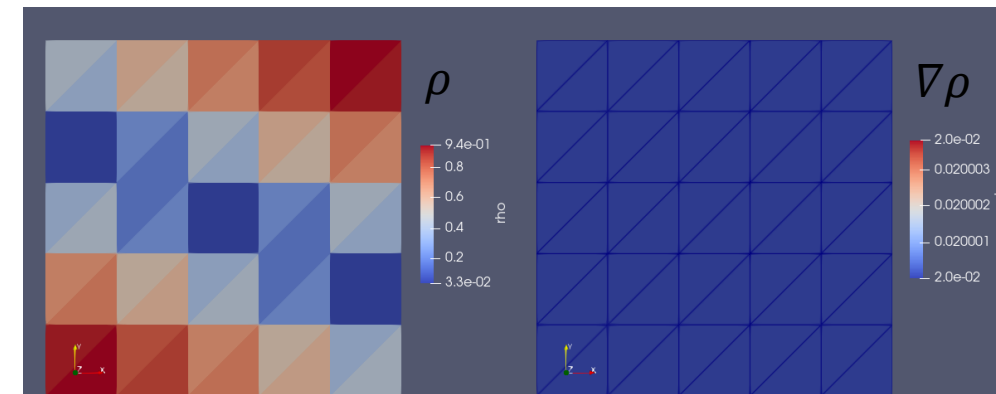
- ‘drho’ makes assemble calculate the derivative for each DOF in the test function space and we thus get the gradient vector components:

$$\text{derivative}((rho-Vf)/V , rho , drho) = \lim_{\tau \rightarrow 0} \frac{d}{d\tau} \left(\frac{\rho + \tau d\rho - V_f}{V} \right) = \frac{d\rho}{V}$$

$$\text{assemble}(grad_g * dx) = \int_{\Omega} d\rho \frac{1}{V} d\Omega \text{ where } d\rho \text{ is a test function.}$$

Evaluating this for each test basis function give the gradient vector components (similar to the common load vector term $\int_{\Omega} v f d\Omega$, where v is the test function).

- This is how to take the derivative of a finite element function with respect to its DOFs in FEniCS.
- FEniCS also have a ‘diff’ function, which can be used for “normal” symbolic differentiation.



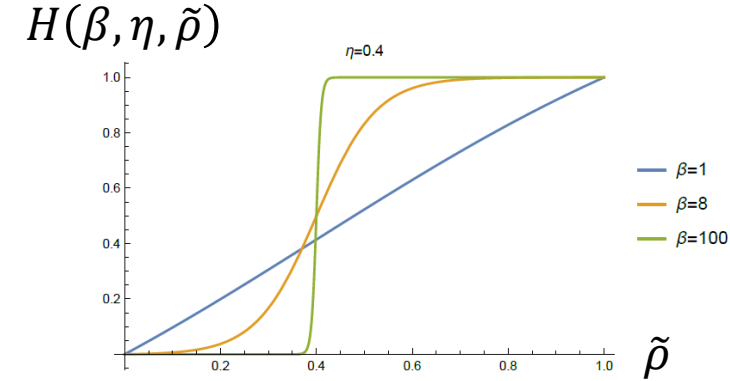
Volume Constraint Gradient

- When we do filtering and projection, the volume constraint is

$$g(\rho) = \frac{1}{V} \int_{\Omega} (\bar{\rho} - V_f) d\Omega \leq 0 \quad \text{where}$$

$$\bar{\rho} = H(\beta, \eta, \tilde{\rho}) = \frac{\tanh(\beta\eta) + \tanh(\beta(\tilde{\rho} - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))} \quad \text{and}$$

$$-r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} - \rho = 0 \quad \text{in } \Omega, \quad \frac{\partial \tilde{\rho}}{\partial \mathbf{n}} = \hat{\mathbf{n}} \cdot \nabla \tilde{\rho} = 0 \quad \text{on } \partial\Omega.$$



- $H(\beta, \eta, \tilde{\rho})$ is just a “normal” function of $\tilde{\rho}$, where $\tilde{\rho}$ is a function of the ρ_i ’s (DOFs of the ρ field), so $\tilde{\rho}(\rho_i)$ and we can simply use the chain-rule and write

$$\frac{d\bar{\rho}}{d\rho_i} = \frac{\partial H(\beta, \eta, \tilde{\rho})}{\partial \tilde{\rho}} \frac{d\tilde{\rho}}{d\rho_i}$$

- $\frac{\partial H(\beta, \eta, \tilde{\rho})}{\partial \tilde{\rho}} \equiv H'(\beta, \eta, \tilde{\rho})$ is found simply by differentiating $H(\beta, \eta, \tilde{\rho})$:

$$H'(\beta, \eta, \tilde{\rho}) = \frac{\beta}{(\tanh(\beta\eta) + \tanh(\beta(1 - \eta))) \cosh^2(\beta(\tilde{\rho} - \eta))}$$

- $\frac{d\tilde{\rho}}{d\rho_i}$ is not as easy, since $\tilde{\rho}$ is found by solving a PDE with all ρ_i ’s as an input.
- Fortunately, it is not as bad as it seems 😊

Volume Constraint Gradient

- We have

$$g(\rho) = \frac{1}{V} \int_{\Omega} (\bar{\rho} - V_f) d\Omega \quad , \quad \bar{\rho} = H(\beta, \eta, \tilde{\rho}), \quad -r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} - \rho = 0 \text{ in } \Omega, \quad \frac{\partial \tilde{\rho}}{\partial \mathbf{n}} = \hat{\mathbf{n}} \cdot \nabla \tilde{\rho}$$

- We want to take the derivative of g , and make sure $-r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} - \rho = 0$, $\hat{\mathbf{n}} \cdot \nabla \tilde{\rho} = 0$ are satisfied.
- We now write

$$g(\rho) = \frac{1}{V} \int_{\Omega} (\bar{\rho} - V_f) d\Omega = \frac{1}{V} \int_{\Omega} (\bar{\rho} - V_f) d\Omega - \int_{\Omega} (-r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} - \rho) v_c d\Omega$$

where v_c is *any* test function in the test function space used for the Helmholtz equation.

- This is possible, since $-r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} - \rho = 0$ *always*, at all spatial locations and all values of ρ .
- v_c can be viewed as a field of Lagrange multipliers.
- The total derivative $\frac{dg}{d\rho_i}$ is now calculated...

Volume Constraint Gradient

- With $g(\rho) = \frac{1}{V} \int_{\Omega} (\bar{\rho} - V_f) d\Omega - \int_{\Omega} (-r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} - \rho) v_c d\Omega$ and $\bar{\rho} = H(\beta, \eta, \tilde{\rho})$, the total derivative is then

$$\begin{aligned}
 \frac{dg}{d\rho_i} &= \frac{1}{V} \int_{\Omega} \frac{\partial H}{\partial \tilde{\rho}} \frac{d\tilde{\rho}}{d\rho_i} d\Omega - \int_{\Omega} \left(-r^2 \nabla^2 \frac{d\tilde{\rho}}{d\rho_i} + \frac{d\tilde{\rho}}{d\rho_i} - \frac{\partial \rho}{\partial \rho_i} \right) v_c d\Omega \\
 &= \frac{1}{V} \int_{\Omega} \frac{\partial H}{\partial \tilde{\rho}} \frac{d\tilde{\rho}}{d\rho_i} d\Omega - \int_{\Omega} \left(-r^2 v_c \nabla \cdot \left(\nabla \frac{d\tilde{\rho}}{d\rho_i} \right) + \frac{d\tilde{\rho}}{d\rho_i} v_c - \frac{\partial \rho}{\partial \rho_i} v_c \right) d\Omega \\
 &= \frac{1}{V} \int_{\Omega} \frac{\partial H}{\partial \tilde{\rho}} \frac{d\tilde{\rho}}{d\rho_i} d\Omega - \int_{\Omega} \left(r^2 \nabla v_c \cdot \left(\nabla \frac{d\tilde{\rho}}{d\rho_i} \right) + \frac{d\tilde{\rho}}{d\rho_i} v_c - \frac{\partial \rho}{\partial \rho_i} v_c \right) d\Omega - \int_{\partial\Omega} \left(-r^2 \hat{n} \cdot \left(\nabla \frac{d\tilde{\rho}}{d\rho_i} \right) v_c \right) dS \\
 &= \frac{1}{V} \int_{\Omega} \frac{\partial H}{\partial \tilde{\rho}} \frac{d\tilde{\rho}}{d\rho_i} d\Omega - \int_{\Omega} \left((-r^2 \nabla^2 v_c + v_c) \frac{d\tilde{\rho}}{d\rho_i} - \frac{\partial \rho}{\partial \rho_i} v_c \right) d\Omega + \int_{\partial\Omega} \left(-r^2 \hat{n} \cdot \nabla v_c \frac{d\tilde{\rho}}{d\rho_i} \right) dS \\
 &= - \int_{\Omega} \left(\left(-r^2 \nabla^2 v_c + v_c - \frac{1}{V} \frac{\partial H}{\partial \tilde{\rho}} \right) \frac{d\tilde{\rho}}{d\rho_i} - \frac{\partial \rho}{\partial \rho_i} v_c \right) d\Omega + \int_{\partial\Omega} \left(-r^2 \hat{n} \cdot \nabla v_c \frac{d\tilde{\rho}}{d\rho_i} \right) dS
 \end{aligned}$$

$$\begin{aligned}
 &\int_{\partial\Omega} \left(-r^2 \hat{n} \cdot \left(\nabla \frac{d\tilde{\rho}}{d\rho_i} \right) v_c \right) dS \\
 &= \int_{\partial\Omega} \frac{\partial}{\partial \rho_i} \left(-r^2 \hat{n} \cdot \nabla \tilde{\rho} v_c \right) dS \\
 &= 0 \\
 &\text{due to the zero Neumann} \\
 &\text{boundary conditions,} \\
 &\hat{n} \cdot \nabla \tilde{\rho} = 0 \text{ on } \partial\Omega
 \end{aligned}$$

- By choosing the test function v_c to satisfy

$$-r^2 \nabla^2 v_c + v_c = \frac{1}{V} \frac{\partial H}{\partial \tilde{\rho}} \quad \text{in } \Omega, \quad \hat{n} \cdot \nabla v_c = 0 \quad \text{on } \partial\Omega$$

the derivative is simply

$$\frac{dg}{d\rho_i} = \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} v_c d\Omega$$

- This is OK to do, as the test function is *any* function in the test function space used for the Helmholtz equation. This particular choice makes it simple to calculate $\frac{dg}{d\rho_i}$.

Volume Constraint Gradient

- Using $\frac{\partial H(\beta, \eta, \tilde{\rho})}{\partial \tilde{\rho}} = H'(\beta, \eta, \tilde{\rho}) = \frac{\beta}{(\tanh(\beta\eta) + \tanh(\beta(1-\eta))) \cosh^2(\beta(\tilde{\rho} - \eta))}$

the procedure is:

Solve first the “adjoint PDE”

$-r^2 \nabla^2 v_c + v_c = \frac{1}{V} H'(\beta, \eta, \tilde{\rho})$ with Neumann boundary conditions $\hat{n} \cdot \nabla v_c = 0$
to obtain v_c , then calculate

$$\frac{dg}{d\rho_i} = \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} v_c d\Omega$$

- This requires us to solve an additional PDE (the “adjoint PDE”).
- From the solution of the adjoint PDE, the derivative can easily be calculated using ‘assemble’ (integrate) and ‘derivative’ in FEniCS:

```
drho=TestFunction(Vd)
drho_drhoi=derivative(rho,rho,drho)
dg_drhoi=assemble(drho_drhoi*vc*dx)
```

See also the function `filter_gradient` in `filter_and_project.py`.

- Note that the adjoint PDE is very similar to the “original” PDE, only the right hands side is different.
- Note also that we solve for the test function v_c , which makes the adjoint PDE linear.

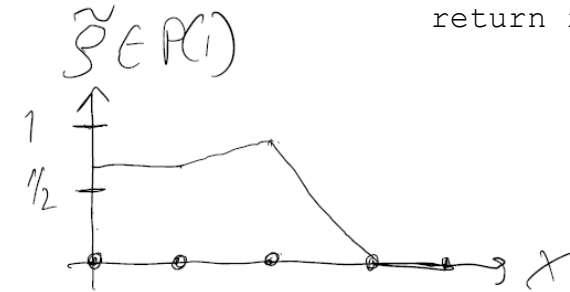
Helmholtz filtering PDE:

$$\begin{aligned} -r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} - \rho &= 0 \text{ in } \Omega, \\ \hat{n} \cdot \nabla \tilde{\rho} &= 0 \text{ on } \partial\Omega. \end{aligned}$$

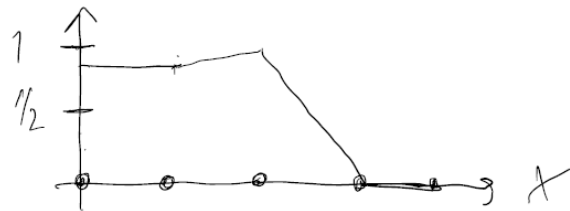
2-stage projection

In the program, we had an additional L^2 projection of $\tilde{\rho}$ before $H(\beta, \eta, \tilde{\rho})$ was L^2 projected.

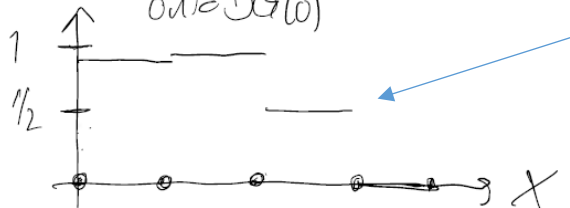
```
def Heavi_project(self, beta, eta, rho_tilde):
    # Projects rho_tilde to Vd and then projects H(u) to Vd
    # Without this 2-stage projection, 1-element gradients from the functions in Vc
    # will create gradients in rho_phys.
    u = self.project_Vd(rho_tilde)
    rho_bar = self.project_Vd(self.Heavi(beta, eta, u))
    return rho_bar, u
```



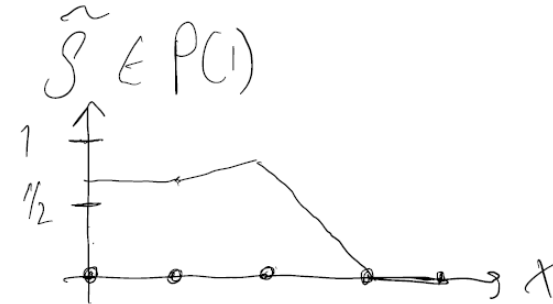
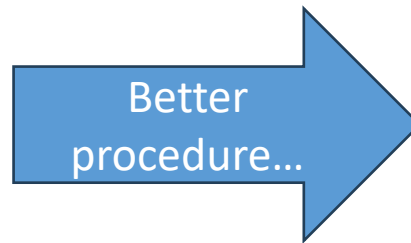
$$u = H(\tilde{\rho})$$



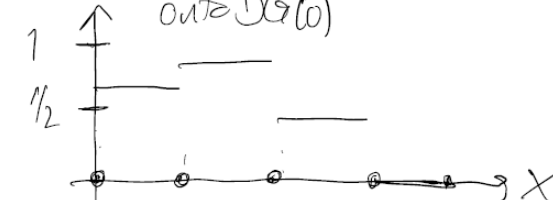
$$\bar{\rho} = L^2 \text{ projection of } u \text{ onto } DG(1)$$



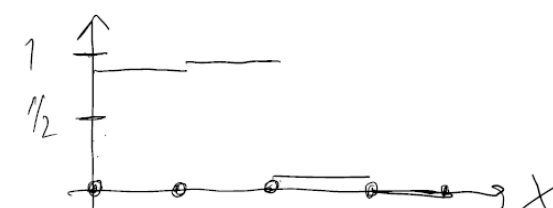
Leaves a
'soft' region
 $\bar{\rho} \approx \frac{1}{2}$ of
1 mesh
element in
size between
material and
void.



$$u = L^2 \text{ projection of } \tilde{\rho} \text{ onto } DG(1)$$



$$\bar{\rho} \in H(u)$$



Allows for
'hard'
transition
between
material and
void.

Volume Constraint Gradient

- In the program, we had an additional L^2 projection of $\tilde{\rho}$ before $H(\beta, \eta, \tilde{\rho})$ was L^2 projected.

```
def Heavi_project(self,beta,eta,rho_tilde):
    # Projects rho_tilde to Vd and then projects H(u) to Vd
    # Without this 2-stage projection, 1-element gradients from the functions in Vc
    # will create gradients in rho_phys.
    u=self.project_Vd(rho_tilde)
    rho_bar=self.project_Vd(self.Heavi(beta,eta,u))
    return rho_bar,u
```

- Repeating the procedure including this extra L^2 projection and considering a general functional on the form

$$g(\rho) = \int_{\Omega} F(\bar{\rho}) d\Omega$$

we get the following procedure to calculate $\frac{dg}{d\rho_i} = \frac{d}{d\rho_i} \int_{\Omega} F(\bar{\rho}) d\Omega$

Solve the adjoint PDE

$$-r^2 \nabla^2 v_c + v_c = \frac{\partial F}{\partial \bar{\rho}} \frac{\partial \bar{\rho}}{\partial u}$$

with Neumann boundary conditions $\hat{n} \cdot \nabla v_c = 0$ and $\frac{\partial \bar{\rho}}{\partial u} = H'(\beta, \eta, u)$.

and u being the L^2 projection of $\tilde{\rho}$ to the function space of ρ to obtain v_c , then calculate the derivative as

$$\frac{dg}{d\rho_i} = \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} v_c d\Omega$$

- This is almost the same as before. Use the definition of an L^2 projection to prove this yourself 😊

Cost function gradient

The same procedure can be used to calculate the gradient of the cost function,

$$f(\rho) = \int_{\Omega} \frac{1}{2} \sigma_{ij}(u) \epsilon_{ij}(u) d\Omega.$$

$$\sigma_{ij,j} = f_i \quad , \quad \sigma_{ij} n_j = T_i \text{ on } \partial\Omega^N \quad , \quad u_i = u_i^D \text{ on } \partial\Omega^D$$

$$\epsilon_{ij} = \frac{1}{2} (u_{i,j} + u_{j,i})$$

$$\sigma_{ij} = \lambda(E) \epsilon_{kk} \delta_{ij} + 2\mu(E) \epsilon_{ij} = \lambda(E) u_{k,k} \delta_{ij} + \mu(E) (u_{i,j} + u_{j,i}) \text{ with } E = E(\rho)$$

The weak form of the linear elastic problem is: Find $u \in V$ such that

$$\int_{\Omega} \sigma_{ij}(u) \epsilon_{ij}(v) d\Omega = \int_{\Omega} f_i v_i d\Omega + \int_{\partial\Omega_T} T_i v_i dS \quad \forall v \in \hat{V}$$

Using a test function for the displacement field, v , we write

$$f = \int_{\Omega} \frac{1}{2} \sigma_{ij}(u) \epsilon_{ij}(u) d\Omega = \int_{\Omega} \frac{1}{2} \sigma_{ij}(u) \epsilon_{ij}(u) d\Omega - \overbrace{\left(\int_{\Omega} \sigma_{ij}(u) \epsilon_{ij}(v) d\Omega - \int_{\Omega} f_i v_i d\Omega - \int_{\partial\Omega_T} T_i v_i dS \right)}^{=0}$$

Assuming, for simplicity, in the following that neither f_i nor T_i depends on ρ (not true for gravitational loading, $f_i \propto \bar{\rho}$), we get

$$\begin{aligned} \frac{df}{d\rho_m} &= \int_{\Omega} \left(\frac{\partial}{\partial u_{k,l}} \left(\frac{1}{2} \sigma_{ij}(u) \epsilon_{ij}(u) \right) \frac{du_{k,l}}{d\rho_m} + \frac{1}{2} \frac{\partial \sigma_{ij}(u)}{\partial E} \epsilon_{ij}(u) \frac{dE}{d\rho_m} \right) d\Omega - \int_{\Omega} \left(\frac{\partial}{\partial u_{k,l}} (\sigma_{ij}(u) \epsilon_{ij}(v)) \frac{du_{k,l}}{d\rho_m} + \frac{\partial \sigma_{ij}(u)}{\partial E} \epsilon_{ij}(v) \frac{dE}{d\rho_m} \right) d\Omega \\ &= \int_{\Omega} \left(\frac{1}{2} \left(\frac{\partial \sigma_{ij}(u)}{\partial u_{k,l}} \epsilon_{ij}(u) + \sigma_{ij}(u) \frac{\partial \epsilon_{ij}(u)}{\partial u_{k,l}} \right) - \frac{\partial \sigma_{ij}(u)}{\partial u_{k,l}} \epsilon_{ij}(v) \right) \frac{du_{k,l}}{d\rho_m} d\Omega + \int_{\Omega} \frac{\partial \sigma_{ij}(u)}{\partial E} \left(\frac{1}{2} \epsilon_{ij}(u) - \epsilon_{ij}(v) \right) \frac{dE}{d\rho_m} d\Omega \end{aligned}$$

From previous slide:

Cost function gradient

$$\begin{aligned} \frac{df}{d\rho_m} &= \int_{\Omega} \left(\frac{\partial}{\partial u_{k,l}} \left(\frac{1}{2} \sigma_{ij}(u) \epsilon_{ij}(u) \right) \frac{du_{k,l}}{d\rho_m} + \frac{1}{2} \frac{\partial \sigma_{ij}(u)}{\partial E} \epsilon_{ij}(u) \frac{dE}{d\rho_m} \right) d\Omega - \int_{\Omega} \left(\frac{\partial}{\partial u_{k,l}} (\sigma_{ij}(u) \epsilon_{ij}(v)) \frac{du_{k,l}}{d\rho_m} + \frac{\partial \sigma_{ij}(u)}{\partial E} \epsilon_{ij}(v) \frac{dE}{d\rho_m} \right) d\Omega \\ &= \int_{\Omega} \left(\frac{1}{2} \left(\frac{\partial \sigma_{ij}(u)}{\partial u_{k,l}} \epsilon_{ij}(u) + \sigma_{ij}(u) \frac{\partial \epsilon_{ij}(u)}{\partial u_{k,l}} \right) - \frac{\partial \sigma_{ij}(u)}{\partial u_{k,l}} \epsilon_{ij}(v) \right) \frac{du_{k,l}}{d\rho_m} d\Omega + \int_{\Omega} \frac{\partial \sigma_{ij}(u)}{\partial E} \left(\frac{1}{2} \epsilon_{ij}(u) - \epsilon_{ij}(v) \right) \frac{dE}{d\rho_m} d\Omega \end{aligned}$$

We will try to make the bracket in the first integral identically zero, such that we do not need $\frac{du_{k,l}}{d\rho_m}$. We have

$$\epsilon_{ij}(u) = \frac{1}{2} (u_{i,j} + u_{j,i}) \quad , \quad \sigma_{ij}(u) = \lambda \epsilon_{nn} \delta_{ij} + 2\mu \epsilon_{ij} = \lambda u_{n,n} \delta_{ij} + \mu (u_{i,j} + u_{j,i})$$

The strain give

$$\frac{\partial \epsilon_{ij}(u)}{\partial u_{k,l}} = \frac{1}{2} (\delta_{ik} \delta_{jl} + \delta_{jk} \delta_{il}) \quad , \quad \frac{\partial \epsilon_{nn}(u)}{\partial u_{k,l}} = \frac{1}{2} (\delta_{nk} \delta_{nl} + \delta_{nk} \delta_{nl}) = \delta_{kl}$$

And the stress

$$\frac{\partial \sigma_{ij}(u)}{\partial u_{k,l}} = \lambda \frac{\partial \epsilon_{nn}(u)}{\partial u_{k,l}} \delta_{ij} + 2\mu \frac{\partial \epsilon_{ij}(u)}{\partial u_{k,l}} = \lambda \delta_{kl} \delta_{ij} + \mu (\delta_{ik} \delta_{jl} + \delta_{jk} \delta_{il})$$

The part inside the bracket in the integrand is then

$$\frac{1}{2} \left(\frac{\partial \sigma_{ij}(u)}{\partial u_{k,l}} \epsilon_{ij}(u) + \sigma_{ij}(u) \frac{\partial \epsilon_{ij}(u)}{\partial u_{k,l}} \right) - \frac{\partial \sigma_{ij}(u)}{\partial u_{k,l}} \epsilon_{ij}(v) = \frac{1}{2} (\lambda \delta_{kl} \epsilon_{ii}(u) + 2\mu \epsilon_{kl}(u) + \sigma_{kl}(u)) - (\lambda \delta_{kl} \epsilon_{ii}(v) + 2\mu \epsilon_{kl}(v)) = \sigma_{kl}(u) - \sigma_{kl}(v)$$

If the Dirichlet boundary conditions are $u_i^D = 0$ (same as v 's DBC), we can simply choose $v = u$ to make the bracket zero.

This is a nice feature of the cost function being the strain energy, due to the fact that the equilibrium equations can be obtained by minimizing the strain energy. The adjoint solution, v , is the same as the displacement, if zero Dirichlet boundary conditions are used. ¹⁵

Cost function gradient

With the adjoint solution being given by $v = u$, we have

$$\frac{df}{d\rho_m} = \int_{\Omega} \frac{\partial \sigma_{ij}(u)}{\partial E} \left(\frac{1}{2} \epsilon_{ij}(u) - \epsilon_{ij}(u) \right) \frac{dE}{d\rho_m} d\Omega = -\frac{1}{2} \int_{\Omega} \frac{\partial \sigma_{ij}(u)}{\partial E} \epsilon_{ij}(u) \frac{dE}{d\rho_m} d\Omega$$

We note that $E(\rho) = E(\bar{\rho}(\rho)) = E_{\min} + (E_{\text{material}} - E_{\min}) \bar{\rho}^p$ and write

$$\frac{df}{d\rho_m} = \frac{d}{d\rho_m} \int_{\Omega} F(\bar{\rho}) d\Omega \quad , \quad F(\bar{\rho}) = -\frac{1}{2} \frac{\partial \sigma_{ij}(u)}{\partial E} \epsilon_{ij}(u) E(\bar{\rho})$$

where only $E(\bar{\rho})$ is to be differentiated when calculating $\frac{\partial F}{\partial \bar{\rho}}$.

This is on the form we had a few slides back, i.e. to calculate the gradient we solve

$$-r^2 \nabla^2 v_c + v_c = \frac{\partial F}{\partial \bar{\rho}} H'(\beta, \eta, \tilde{\rho}_{L^2}) \quad , \quad \hat{n} \cdot \nabla v_c = 0$$

with $\tilde{\rho}_{L^2}$ being the L^2 projection of $\tilde{\rho}$ onto the piecewise constant function space ($\tilde{\rho}_{L^2}$ was named u earlier). Then calculate

$$\frac{df}{d\rho_m} = \int_{\Omega} \frac{\partial \rho}{\partial \rho_m} v_c d\Omega$$

For this to work, the right hands side of the filter equation needs to be in the function space we used for ρ and $\bar{\rho}$, but

$F(\bar{\rho}) = -\frac{1}{2} \frac{\partial \sigma_{ij}(u)}{\partial E} \epsilon_{ij}(u) E(\bar{\rho})$ has elements from both the function space used for the displacement (actually the spatial derivative of these functions) and the function space used for $\bar{\rho}$ (the same as for ρ , piecewise constant functions).

It is thus necessary to do a L^2 projection of F onto the function space used for $\bar{\rho}$ before passing F to the filter equation.

A few slides ago....

Repeating the procedure including this extra L^2 projection and considering a general functional on the form

$$g(\rho) = \int_{\Omega} F(\bar{\rho}) d\Omega$$

we get the following procedure to calculate $\frac{dg}{d\rho_i} = \frac{d}{d\rho_i} \int_{\Omega} F(\bar{\rho}) d\Omega$

Solve the adjoint PDE

$$-r^2 \nabla^2 v_c + v_c = \frac{\partial F}{\partial \bar{\rho}} \frac{\partial \bar{\rho}}{\partial u}$$

with Neumann boundary conditions $\hat{n} \cdot \nabla v_c = 0$ and

$$\frac{\partial \bar{\rho}}{\partial u} = H'(\beta, \eta, u).$$

and u being the L^2 projection of $\tilde{\rho}$ to the function space of ρ to obtain v_c , then calculate the derivative as

$$\frac{dg}{d\rho_i} = \int_{\Omega} \frac{\partial \rho}{\partial \rho_i} v_c d\Omega$$

Gradients

- As can be seen, the adjoint method can efficiently calculate the required gradients.
- The computational requirements is about the same (or smaller) than solving the forward model.
- With the strain energy as the cost function, solution of the adjoint model for the mechanical equilibrium (with zero Dirichlet boundary conditions) is the already known displacements, so we only need to solve one mechanical problem to get both the cost function and its gradient.
- The method works also for non-linear problems.
- For non-linear problems, the adjoint model is linear and requires less work than the forward model.
 - The adjoint PDE is solved for the test function of the original PDE.
 - The test function is introduced to obtain the weak form, by first multiplication followed by integration.
 - These operations always yield a linear PDE in the test function.
- One can also use the adjoint method for time-dependent problems, which are in fact just a series of “static” problem to advance the solution to the next time.
- The adjoint equation corresponds here to solving a problem starting from the final time and ending at the initial time, i.e., “backwards in time”.
- While the adjoint method as presented here is very nice and efficient, it is often a bit time-consuming to derive all the equations and implement all steps appropriately....

pyadjoint

- pyadjoint automates the task of calculating gradients using the adjoint method.
- pyadjoint was previously named 'dolphin-adjoint'.
<http://www.dolphin-adjoint.org/en/latest/>
- The following paper give a good introduction to dolphin-adjoint. Many things still hold up for pyadjoint, but the new implementation is entirely written in python and does not use a 'libadjoint'.
Patrick E. Farrell, David A. Ham, Simon W. Funke and Marie E. Rognes (2013).
Automated derivation of the adjoint of high-level transient finite element programs,
SIAM Journal on Scientific Computing 35.4, pp. C369-C393.
http://www.dolphin-adjoint.org/en/latest/_static/dolphin_adjoint.pdf
- The page
<http://www.dolphin-adjoint.org/en/latest/documentation/maths/index.html#dolphin-adjoint-mathematical-background>
give a good overall introduction to the mathematical background of adjoints.
- We will now look at the idea in pyadjoint/dolphin-adjoint and how we use this for our topology optimization, so we do not have to explicitly deal with implementing adjoint models ourselves.
- Dolphin-adjoint was awarded the Winkinson prize for numerical software in 2015:
https://en.wikipedia.org/wiki/J._H._Wilkinson_Prize_for_Numerical_Software#2015

pyadjoint

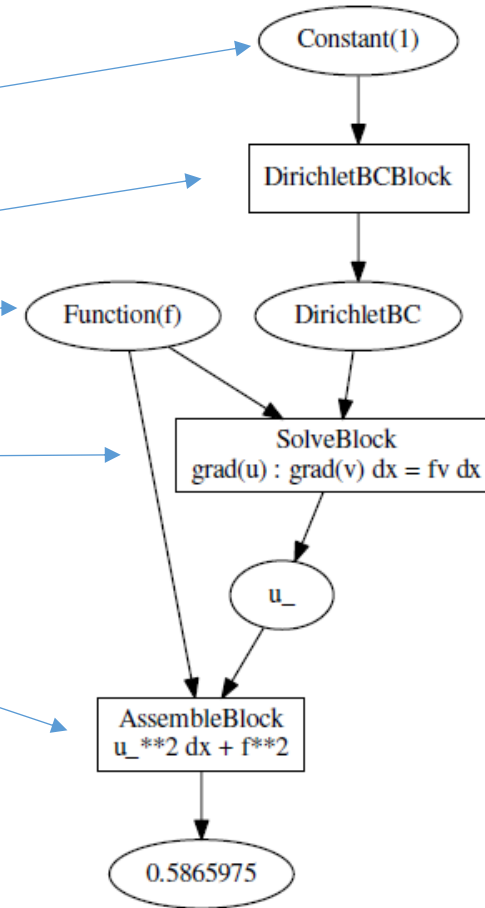
- pyadjoint consists of a main module named pyadjoint, which is a generic module for deriving adjoint models.
- A module named fenics_adjoint (the name 'dolfin_adjoint' can also be used and just “links” to fenics_adjoint), which overloads most fenics operations.
- When fenics_adjoint is imported, many call to fenics functions then actually goes to a fenics_adjoint function with the same name.
- The fenics_adjoint function records the call on a “tape”, sets up the data needed to compute gradients and calls the “real” fenics function.
- This overloading thus makes it almost transparent for the programmer to use fenics_adjoint.
- Due to the overloading, one should first import fenics and then fenics_adjoint:
 - from fenics import *
 - from fenics_adjoint import *

pyadjoint

- An example (from https://drive.google.com/file/d/1NjIFj07u_QMfuXB2Z8uv5f2LUDwY1XeM/view?usp=sharing)

```
from fenics import *
from fenics_adjoint import *
# ...
c = Constant(1)
bc = DirichletBC(V, c, "on_boundary")
a = inner(grad(u), grad(v))*dx
L = f*v*dx
solve(a == L, u_, bc)
z = assemble((u_**2 + f**2)*dx)
dzdc = compute_gradient(z, c)
```

- pyadjoint records the graph as the code is executed.
- When the gradient is to be computed, the graph is used to derive an efficient model for the gradient computation.



pyadjoint has a `'tape.visualize()'` function to show these graphs, but I could not get it working with the latest tensorflow package (required)....

pyadjoint

- This is very clever and almost transparent to the programmer!
 - The adjoint model is automatically build using the formulation used to state the forward model.
 - Solvers for the adjoint model are automatically generated from the forward model solvers.
 - If the forward model is modified, the adjoint model will automatically be updated.
- One problem, often encountered in practice:
 - The recorded graph must be a representation of the forward model, for pyadjoint to be able to derive the adjoint gradient computation model.
 - If some forward solution steps are made but not recorded, the computed gradients will be WRONG!!
 - This can happen is some methods are used which are not overloaded by fenics_adjoint.
 - Commonly used methods which are **not** overloaded are **all operations which directly manipulate the FEniCS function's vectors/numpy arrays**, e.g.
f.vector().set_local(my_numpy_array)
 - **Don't use these operations** in the forward model, find instead an appropriate FEniCS method which *is* overloaded by fenics_adjoint, e.g.
f.assign(my_function)

pyadjoint

pyadjoint code: ~/miniconda3/envs/fenics2019/lib/python3.12/site-packages/fenics_adjoint/lu_solver.py

```
import backend
from pyadjoint.tape import annotate_tape, get_working_tape
from dolfin_adjoint_common import compat

from .blocks import LUSolveBlock, LUSolveBlockHelper

compat = compat.compat(backend)

class LUSolver(backend.LUSolver):
    def __init__(self, *args, **kwargs):
        backend.LUSolver.__init__(self, *args, **kwargs)

        A = kwargs.pop("A", None)
        method = kwargs.pop("method", "default")

        next_arg_idx = 0
        if len(args) > 0 and isinstance(args[0], compat.MatrixType):
            A = args[0]
            next_arg_idx = 1
        elif len(args) > 1 and isinstance(args[1], compat.MatrixType):
            A = args[1]
            next_arg_idx = 2

        if len(args) > next_arg_idx and isinstance(args[next_arg_idx], str):
            method = args[next_arg_idx]

        self.operator = A
        self.method = method
        self.solver_parameters = {}
        self.block_helper = LUSolveBlockHelper()

    def set_operator(self, arg0):
        self.operator = arg0
        self.block_helper = LUSolveBlockHelper()
        return backend.LUSolver.set_operator(self, arg0)
```

```
def solve(self, *args, **kwargs):
    annotate = annotate_tape(kwargs)

    if annotate:
        if len(args) == 3:
            block_helper = LUSolveBlockHelper()
            A = args[0]
            x = args[1]
            b = args[2]
        elif len(args) == 2:
            block_helper = self.block_helper
            A = self.operator
            x = args[0]
            b = args[1]

        u = x.function
        parameters = self.parameters.copy()

        tape = get_working_tape()
        sb_kwargs = LUSolveBlock.pop_kwargs(kwargs)
        block = LUSolveBlock(A, x, b,
                             lu_solver_parameters=parameters,
                             block_helper=block_helper,
                             lu_solver_method=self.method,
                             **sb_kwargs)

        tape.add_block(block)

    out = backend.LUSolver.solve(self, *args, **kwargs)

    if annotate:
        block.add_output(u.create_block_variable())

    return out
```

Topology optimization code from last week

```
def setUpFunctionals(self,beta,eta):
    from pyadjoint.tape import annotate_tape
    while not annotate_tape():
        continue_annotation() # Run continue until
                               # we actually start annotating...

    tape=get_working_tape()
    tape.clear_tape()
    # Run forward model to record on tape
    (u,rho_bar,rhoFil)=self.Forward(beta,eta)
    eps=self.mech.epsilon(u)
    sig=self.mech.sigma(u,eps)
    # Using annotate=False to not record this step,
    # such that J0 does not depend on rho
    if beta==1:
        self.J0=float(assemble(0.5*inner(sig,eps)*dx,annotate=False))
    # Make functionals
    m=Control(self.rho)
    self.J=0.5*inner(sig,eps)/self.J0*dx
    self.V=(rho_bar-self.Volfrac)/self.VolMax*dx
    J=assemble(self.J)
    V=assemble(self.V)
    #self.Jhat = ReducedFunctional(J,m,tape=tape)
    #self.Vhat = ReducedFunctional(V,m,tape=tape)
    # Copy tape, to be able to run optimize such that e.g.
    # mechanics is not solved for when calculating volume.
    tapeJ=tape.copy()
    tapeV=tape.copy()
    self.Jhat = ReducedFunctional(J,m,tape=tapeJ)
    self.Vhat = ReducedFunctional(V,m,tape=tapeV)
    tapeJ.optimize(controls=[m],functionals=[J])
    tapeV.optimize(controls=[m],functionals=[V])
    while annotate_tape():
        pause_annotation() # Run pause until we
                           # actually pause annotating...
```

```
def Forward(self,beta,eta):
    rho_tilde=self.FP.filter(self.rho)
    rho_bar,_=self.FP.Heavi_project(beta,eta,rho_tilde)
    E=self.Emin+pow(rho_bar,self.pen)*(self.E0-self.Emin)
    self.mech.E=E
    self.mech.mask=rho_bar
    u=self.mech.solve()
    return (u,rho_bar,rho_tilde)
```

```
def f(self,x):
    self.rho.vector().set_local(x.getArray()[:])
    self.rho.vector().apply('')
```

```
J=self.Jhat(self.rho)
V=self.Vhat(self.rho)
return np.array([J,V])
```

```
def g(self,x):
    self.rho.vector().set_local(x.getArray()[:])
    self.rho.vector().apply('')
    dc=self.Jhat.derivative().vector().get_local()
    dv=self.Vhat.derivative().vector().get_local()
    return np.array([dc,dv])
```

pyadjoint function

Creates the functional $\hat{J}(\rho) = J(u(\bar{\rho}(\tilde{\rho}(\rho))))$

Creates the functional $\hat{V}(\rho) = V(\bar{\rho}(\tilde{\rho}(\rho)))$

Verifying the calculated gradients

- Using finite differences: For some input ρ_0
 - Calculate the gradient using the adjoint method as just described.
 - Calculate all or some of the gradient component using finite difference:
 - For component number i : $\frac{dg}{d\rho_i} \approx \frac{g(\rho_0 + h\Delta\rho_i) - g(\rho_0)}{h}$, where $\Delta\rho_i$ is a vector with all zeros except at component i where it takes the value 1. h is a small number.
 - Compare the two gradient components.
- It can sometimes be difficult to spot errors using the finite difference test as:
 - For a problem with many variables, it is only feasible to tests some of the components.
 - It can be hard to judge if h is too large or too small.
 - It can be hard to see if the components are “similar enough”.
- Large error will, however, easily show up, e.g., a wrong sign, a factor of 2 etc.

Verifying the calculated gradients

- Using a Taylor series:
 - Expanding the function in a Taylor series: $g(\rho_0 + h\Delta\rho) = g(\rho_0) + \nabla g(\rho_0) \cdot (h\Delta\rho) + O(h^2)$ for some small number h . Then
$$|g(\rho_0 + h\Delta\rho) - g(\rho_0) - \nabla g(\rho_0) \cdot (h\Delta\rho)| = 0 + O(h^2)$$
 - For a set of small fixed numbers $h = h_0, \frac{h_0}{2}, \frac{h_0}{4}, \dots$ and a field perturbation (vector) $\Delta\rho$, calculate
$$\text{error}(h) = |g(\rho_0 + h\Delta\rho) - g(\rho_0) - \nabla g(\rho_0) \cdot (h\Delta\rho)|$$
 - Observe how $\text{error}(h) \rightarrow 0$ as $h \rightarrow 0$:
 - If g is a non-linear function, the value should decrease by a factor of ~ 4 when halving h .
 - If g is a linear function, the Taylor expansion is exact, and the error should be close to 0 (to machine precision) for any value of h .
- Both the finite difference and the Taylor tests are quite useful when testing your gradients.
- *It is very important that your gradients are well tested before solving your optimization problem, as wrong gradient leads to wrong solution.*
- The `mma.py` PETSc version has functions for testing, named `testGradsTaylor` and `testGrads` for the Taylor test and the finite difference test, respectively.